



SmartTraffic

Arquitectura

Zero Trust en

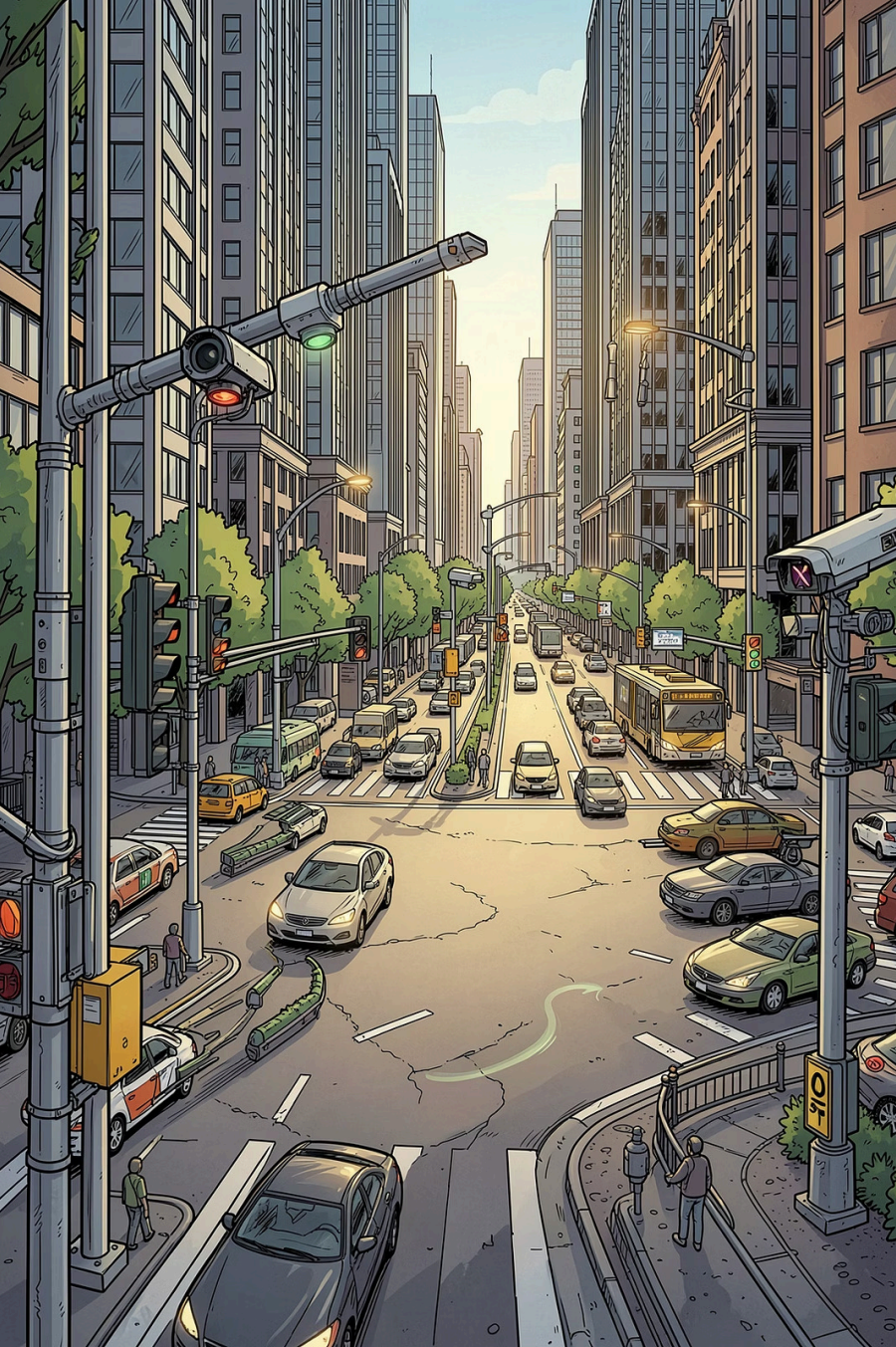
AWS con

nftables y JWT

Juan Camilo Martínez

Jonar Andrés Martínez

Juan Maya



¿Para qué sirve esta arquitectura?

El Problema

El monitoreo de tráfico urbano en tiempo real expone la red a ataques internos. Un nodo comprometido puede propagarse lateralmente sin restricciones.

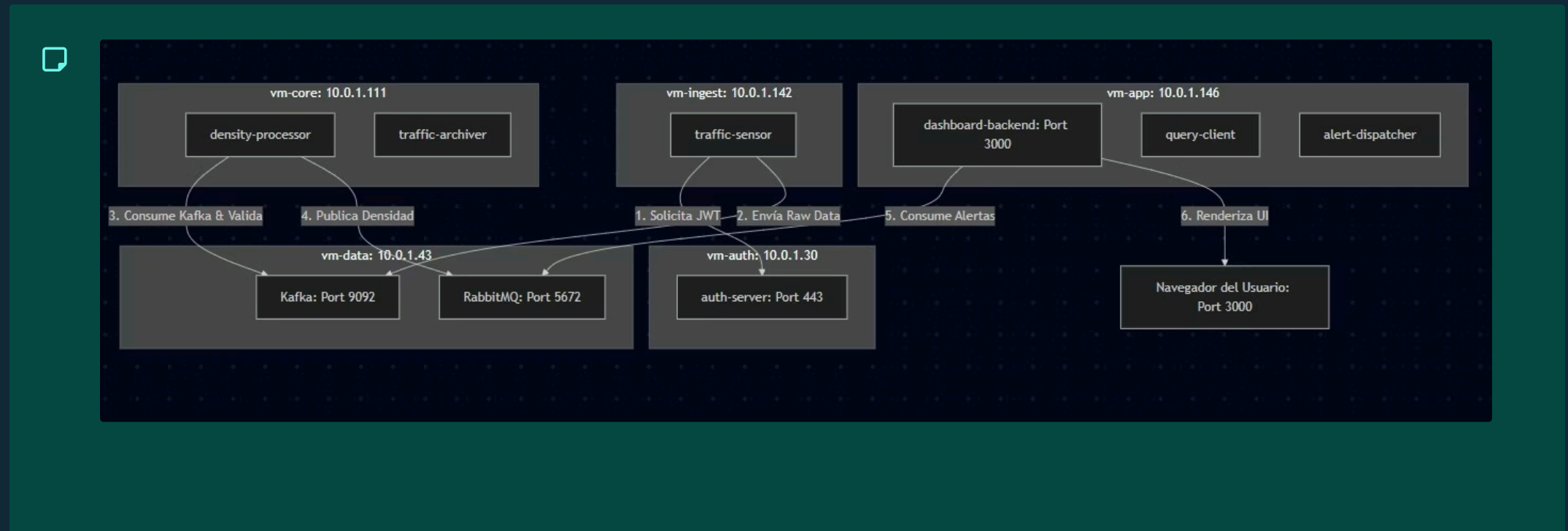
La Solución

Sistema distribuido donde **ningún componente confía en otro por defecto**. Cada VM filtra con nftables; cada petición se autentica con JWT.

Beneficios Clave

Aislamiento total entre nodos, autenticación obligatoria por token y cumplimiento estricto de políticas de seguridad por segmento de red.

Diagrama de Arquitectura



La arquitectura distribuye responsabilidades en cinco nodos aislados por red privada. El tráfico fluye de forma unidireccional: **ingest** → **data** → **core** → **app**, con **vm-auth** como proveedor central de tokens JWT para todos los servicios.

vm-ingest

10.0.1.10 — Sensores de tráfico

vm-auth

10.0.1.20 — Emisor JWT

vm-data

10.0.1.30 — Kafka + RabbitMQ

vm-core

10.0.1.40 — Procesadores

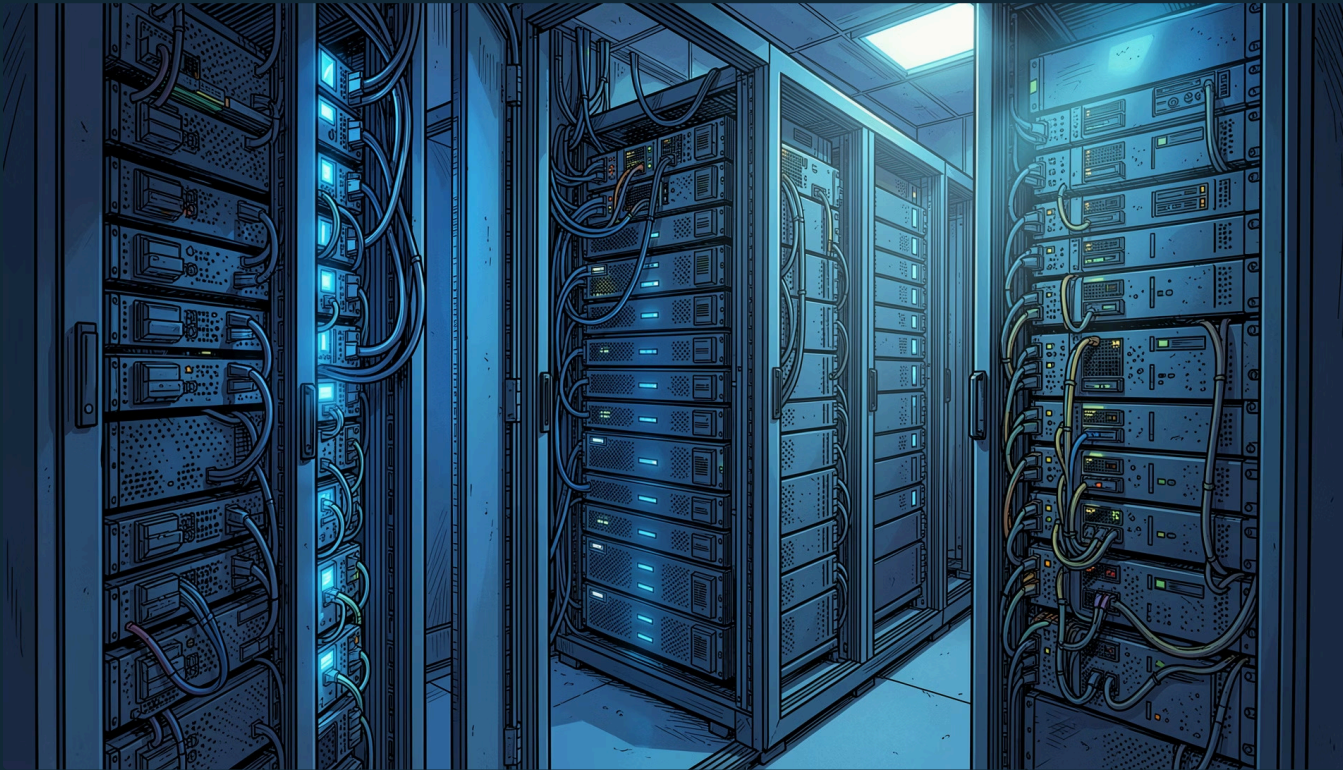
vm-app

10.0.1.50 — Dashboard

Tabla de Zonas de Seguridad

VM	IP Privada	Puertos	Servicio Principal	Acceso Permitido Desde
vm-ingest	10.0.1.10	22, 9092	Sensores de tráfico	vm-data (10.0.1.30)
vm-auth	10.0.1.20	22, 443	Auth Server (JWT)	Todas las VMs del clúster
vm-data	10.0.1.30	22, 9092, 5672	Kafka + RabbitMQ	vm-ingest, vm-core, vm-app
vm-core	10.0.1.40	22, 9092, 5672	Procesadores de datos	vm-data (10.0.1.30)
vm-app	10.0.1.50	22, 3000	Dashboard React	Internet (3000), vm-data

Política Zero Trust con nftables



¿Por qué nftables?

Sucesor moderno de iptables: sintaxis unificada, mejor rendimiento y soporte nativo para conjuntos y mapas. Integración directa con el kernel Linux.

Política Global: `policy drop`

- Traffic entrante bloqueado por defecto en todas las VMs.
- Solo se permite: `loopback`, `ct state established` y SSH.
- Puertos de servicio abiertos únicamente desde IPs de origen explícitas.

i Principio rector: **nada entra a menos que esté explícitamente permitido.**

Fragmentos Críticos de Firewall

vm-auth.nft — Puerto 443 restringido al clúster

```
tcp dport 443 ip saddr {  
  10.0.1.10, 10.0.1.30,  
  10.0.1.40, 10.0.1.50 } accept
```

Mínimo privilegio: Solo las IPs del clúster acceden al emisor JWT.

vm-data.nft — Segmentación Kafka / RabbitMQ

```
tcp dport 9092 ip saddr  
  10.0.1.10 accept # Kafka ← ingest  
tcp dport 9092 ip saddr  
  10.0.1.40 accept # Kafka ← core  
tcp dport 5672 ip saddr  
  10.0.1.40 accept # RabbitMQ ← core  
tcp dport 5672 ip saddr  
  10.0.1.50 accept # RabbitMQ ← app
```

Mínimo privilegio: Kafka y RabbitMQ separados por origen.

vm-app.nft — Solo puerto 3000 expuesto

```
tcp dport 3000 accept #  
Dashboard público  
# El resto cae en policy  
drop automáticamente
```

Mínimo privilegio: Único puerto abierto al exterior; todo lo demás bloqueado.

Orden de Despliegue — Runbook

El orden es **estricto**. Aplicar nftables antes de levantar contenedores puede romper la conectividad NAT de Docker.

O1

vm-auth — Auth Server

```
docker compose up -d  
nft -f /etc/nftables/vm-auth.nft  
sudo systemctl restart docker
```

O2

vm-data — Kafka + RabbitMQ

```
docker compose up -d  
nft -f /etc/nftables/vm-data.nft  
sudo systemctl restart docker
```

O3

vm-core — Procesadores

```
docker compose up -d  
nft -f /etc/nftables/vm-core.nft  
sudo systemctl restart docker
```

O4

vm-ingest — Sensores

```
docker compose up -d  
nft -f /etc/nftables/vm-ingest.nft  
sudo systemctl restart docker
```

O5

vm-app — Dashboard

```
docker compose up -d  
nft -f /etc/nftables/vm-app.nft  
sudo systemctl restart docker
```

⚠ `systemctl restart docker` es necesario porque nftables puede interferir con las cadenas NAT que Docker gestiona automáticamente.

Resolución de Conflictos: Docker vs nftables

El Problema

El comando `nft flush ruleset` elimina las reglas NAT generadas por Docker.

Síntoma: contenedores Up pero sin conectividad entre ellos ni hacia el exterior.

Solución Paso a Paso

1. Aplicar reglas Zero Trust

```
sudo nft -f /etc/nftables/vm-*.nft
```

2. Reiniciar Docker para regenerar NAT

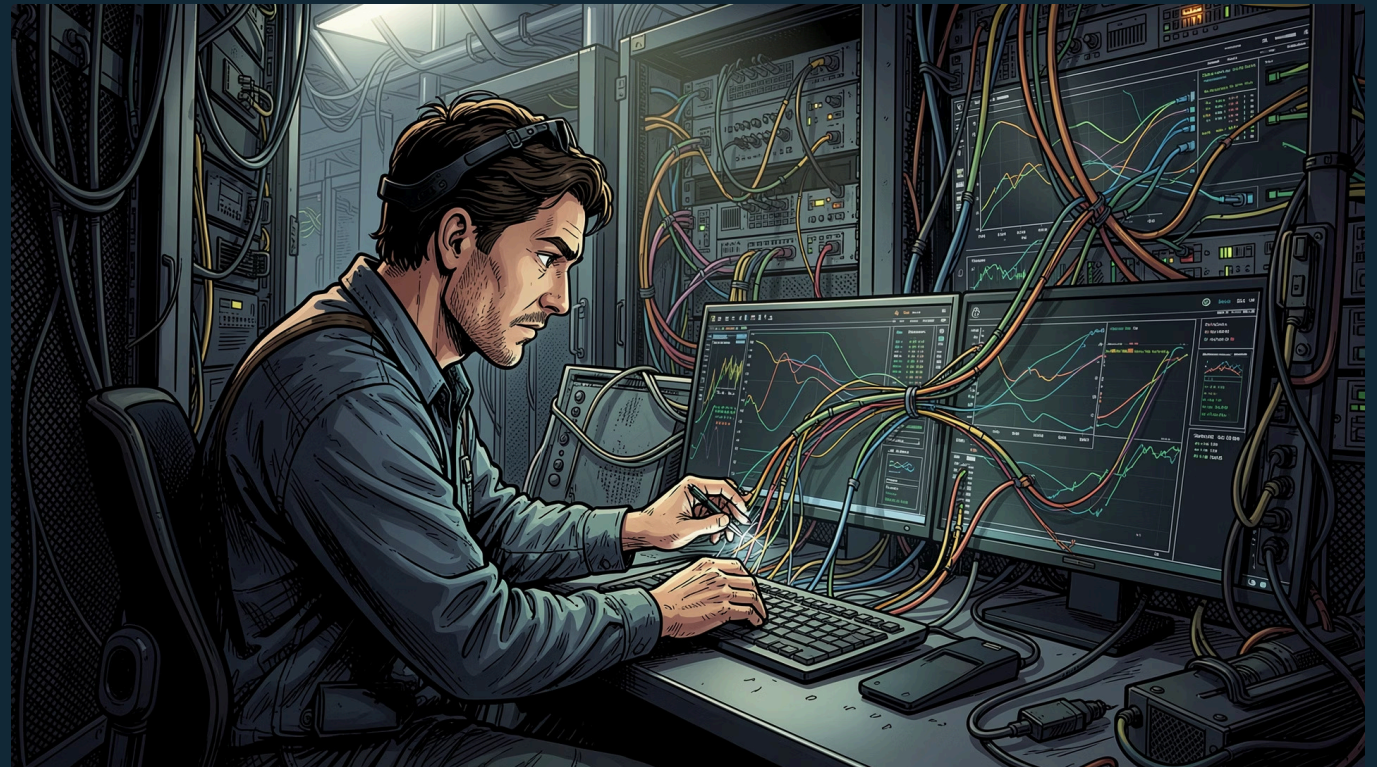
```
sudo systemctl restart docker
```

3. Verificar reglas NAT de Docker

```
sudo nft list table nat
```

4. Confirmar conectividad entre contenedores

```
docker exec vm-core ping vm-data
```



Comandos de Validación y Auditoría

Ver reglas activas

```
sudo nft list ruleset
```

Confirma que `policy drop` está activo y los puertos permitidos son los esperados.

Probar conectividad

```
nc -zv 10.0.1.20 443 #
```

VEHÍCULOS PROCESADOS
418

VELOCIDAD PROMEDIO
44 km/h

ZONAS EN CONGESTIÓN
1

📍 Estado de las Zonas de Tráfico

Zona A FLUIDO

🚗 Vehículos	10
🕒 Velocidad	65 km/h
🕒 Última Act.	3:21:37 PM

Zona B CONGESTIONADO

🚗 Vehículos	99
🕒 Velocidad	10 km/h
🕒 Última Act.	3:21:47 PM

Zona C FLUIDO

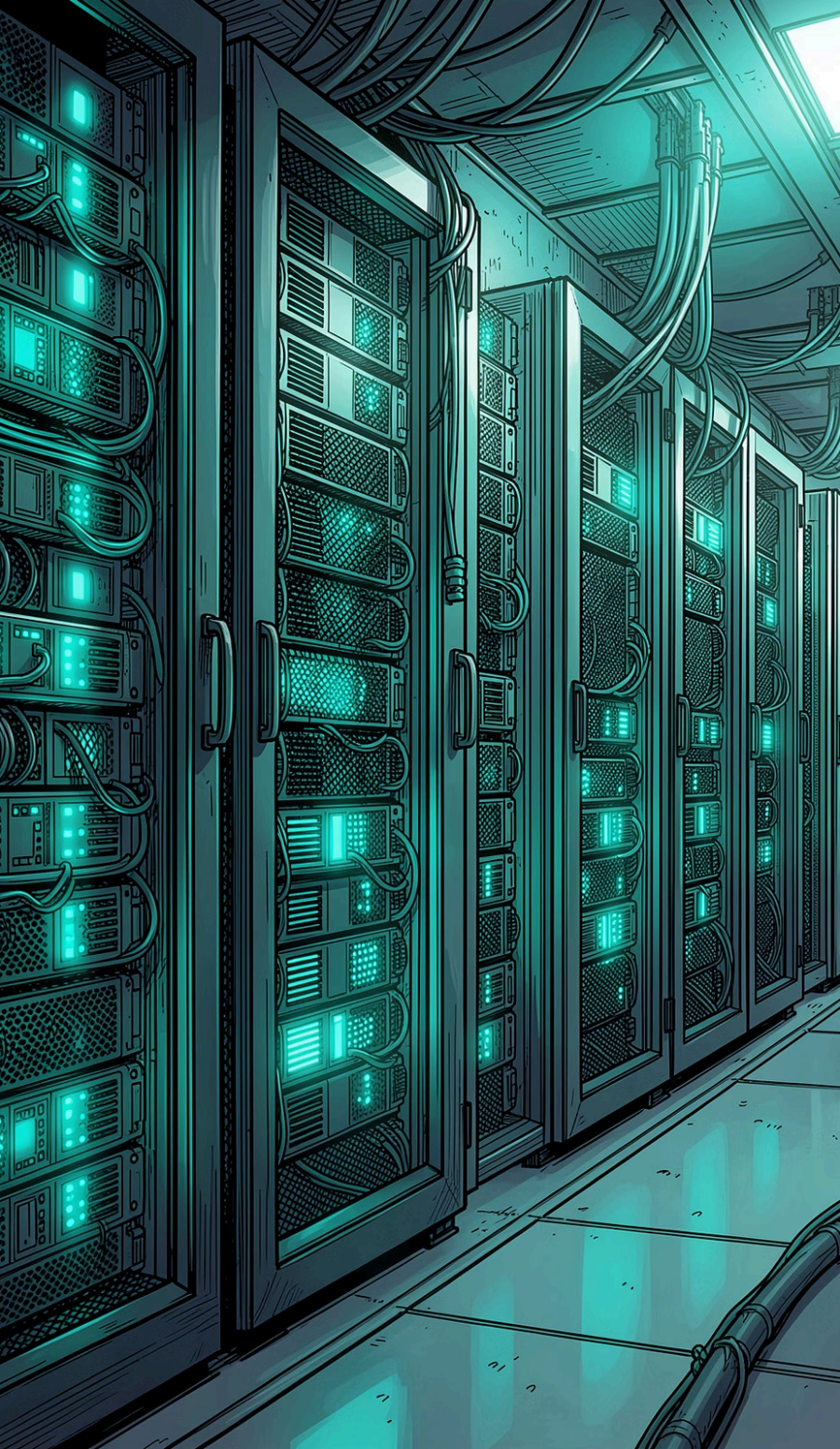
🚗 Vehículos	17
🕒 Velocidad	87 km/h
🕒 Última Act.	3:21:32 PM

Zona D FLUIDO

🚗 Vehículos	22
🕒 Velocidad	83 km/h
🕒 Última Act.	3:21:42 PM

>_ Terminal Eventos RabbitMQ

```
rabbitmq-listener@zero-trust
[3:21:22 PM] Recibido evento Zona D |
Vehiculos: 3 | Velocidad: 63 km/h | Densidad:
FLUIDO
[3:21:27 PM] Recibido evento Zona A |
Vehiculos: 78 | Velocidad: 12 km/h | Densidad:
CONGESTIONADO
[3:21:32 PM] Recibido evento Zona C |
Vehiculos: 17 | Velocidad: 87 km/h | Densidad:
FLUIDO
[3:21:37 PM] Recibido evento Zona A |
Vehiculos: 10 | Velocidad: 65 km/h | Densidad:
FLUIDO
[3:21:42 PM] Recibido evento Zona D |
Vehiculos: 22 | Velocidad: 83 km/h | Densidad:
FLUIDO
[3:21:47 PM] Recibido evento Zona B |
Vehiculos: 99 | Velocidad: 10 km/h | Densidad:
CONGESTIONADO
[3:22:00 PM] Desconectado del WebSocket
backend. Intentando reconectar ...
```



Conclusión y Recomendaciones

Modelo Zero Trust Aplicado

Cada VM opera como una zona de seguridad independiente. Sin confianza implícita, sin movimiento lateral posible. Segmentación real a nivel de kernel.

¿Qué se logra?

Contención de brechas, superficie de ataque mínima, cumplimiento de políticas de seguridad estrictas y auditoría granular del tráfico entre servicios.

Próximos Pasos a Producción

- Monitoreo centralizado de logs de nftables (Falco / Wazuh).
- Rotación corta de JWT (≤ 15 min) con refresh tokens.
- Automatización del runbook con Ansible o Terraform.
- Auditorías periódicas de reglas con `nft list ruleset`.